图形学物理仿真 Tutorial for Lab 0

Part 1: 课程实践（Lab 和 Project） 简介

欢迎大家选修图形学物理仿真课程。我们这学期共需要大家完成紧密围绕课程内容的 3 个 Lab，分别对应这门课程的三大主题：

1. 刚体
2. 流体
3. 弹性体

我们的课程 Lab **不是代码填空**，大家需要自行设计代码结构，从新建文件开始实现模拟部分，因此希望同学们尽早动手。

除此之外，大家也会在学期末完成一项课程 Project。我们会提供一些 Project 题目供大家挑选，同时也很期待看到大家在课堂上找到自己在物理模拟领域感兴趣的方向，并以此作为你的课程 Project。

本学期的 Lab 基于 vcx 代码库，大家可以去可视计算与交互课程了解更多关于代码库的应用。

我们的 Lab 支持 Windows, MacOS 和 Linux 的大多数发行版。如果你需要关于 Lab 的任何帮助，欢迎联系课程助教。

Lab 0: Build the Codebase

1 - Merge Handout Packs

课程lab 0 提供的压缩包解压将建立一个名为 `vcx-sim-master` 的目录：这就是你这个学期做所有 Lab 的目录。 首先你需要按照 Git 官网 的指引安装 Git。

2 - Prepare Compiler

我们的 Lab 需要一个支持 C++20 标准的编译器。推荐使用以下编译器：

- Visual Studio 2019 以上版本，推荐 Visual Studio 2022
- GCC 10 以上版本，推荐 GCC 12
- XCode 13 以上版本
- Clang 13 以上版本

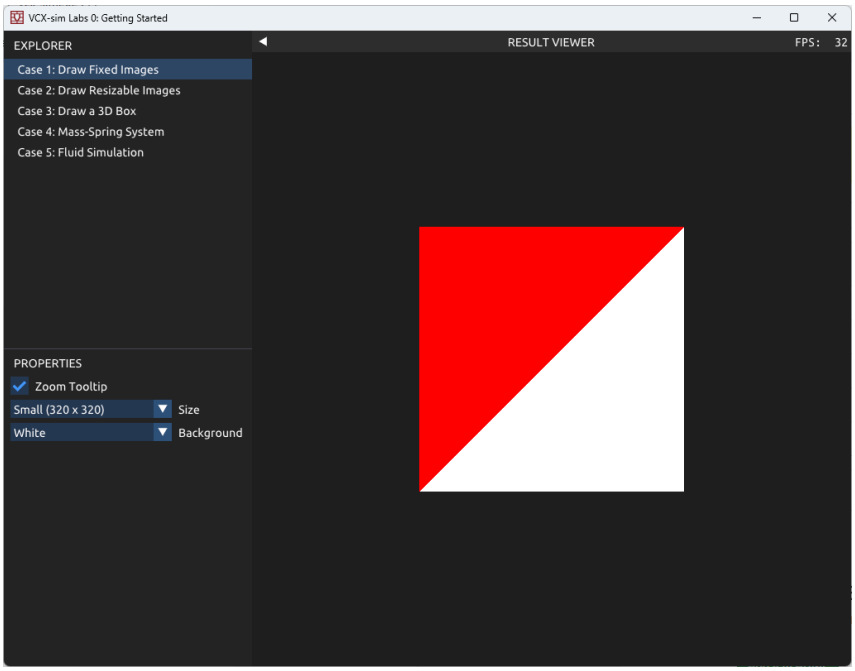
建议使用最新版本的相应编译器。

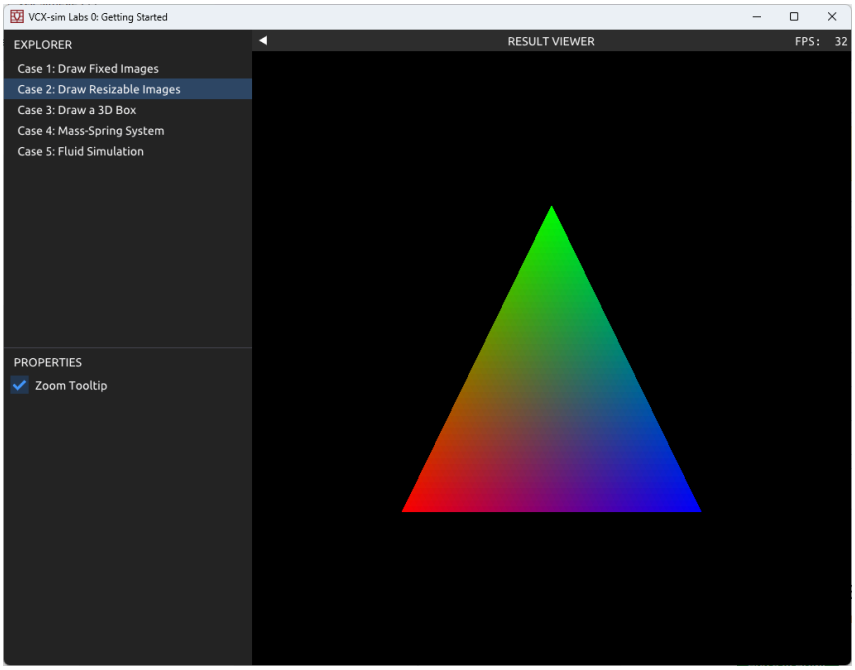
3 - Prepare Git and xmake

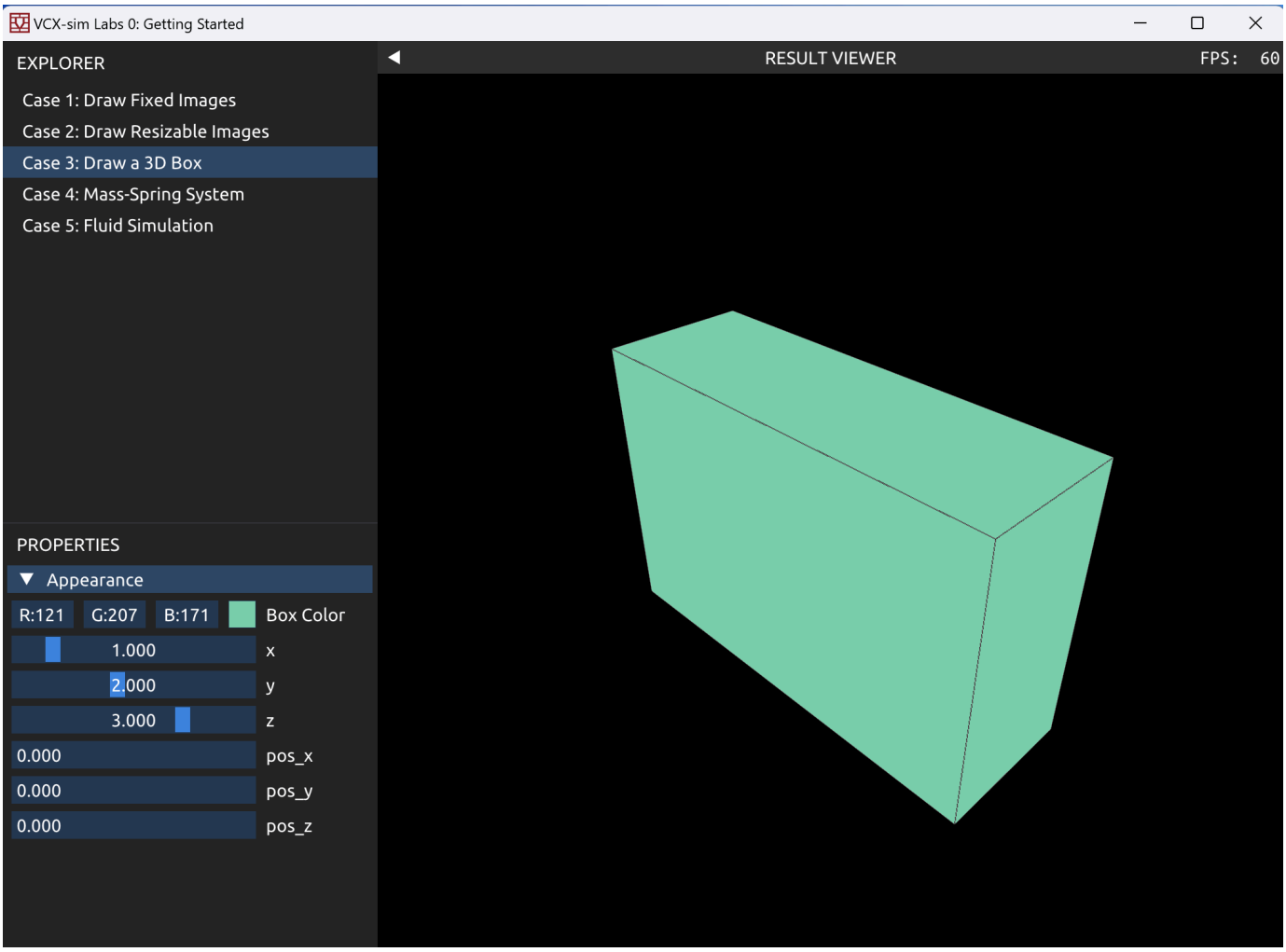
我们的 Lab 使用 xmake (Home) 作为构建工具，而 xmake 依赖 Git 完成包管理等核心功能，请确保已经安装了 Git 。通过 xmake安装说明 中对于你的平台的描述，安装 xmake。完成安装后，可以在终端中运行 `xmake --version` 来确认安装正确。

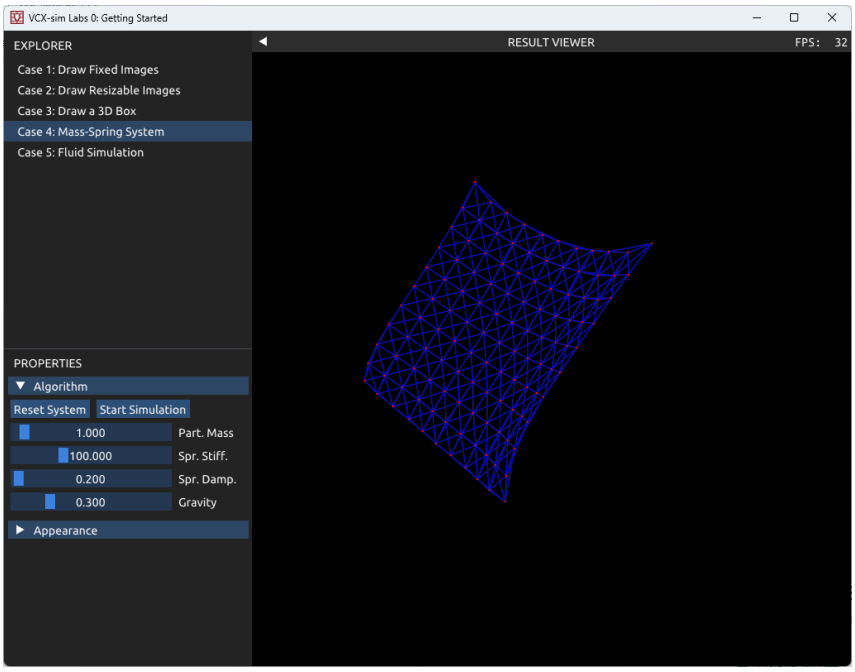
接下来，你只需要在终端中进入 `vcx-sim-master` 目录（下同），然后在命令行中输入 `xmake` 并执行，你就会看到 xmake 自动识别你的平台，下载所有依赖库并完成编译和链接；注意 Windows 下使用 gitbash 终端或者 Anaconda Powershell Prompt 可能无法正常编译，请使用 Windows PowerShell；注意这里可能遇到一些网络问题，可以参见FAQ 中网络错误的部分。

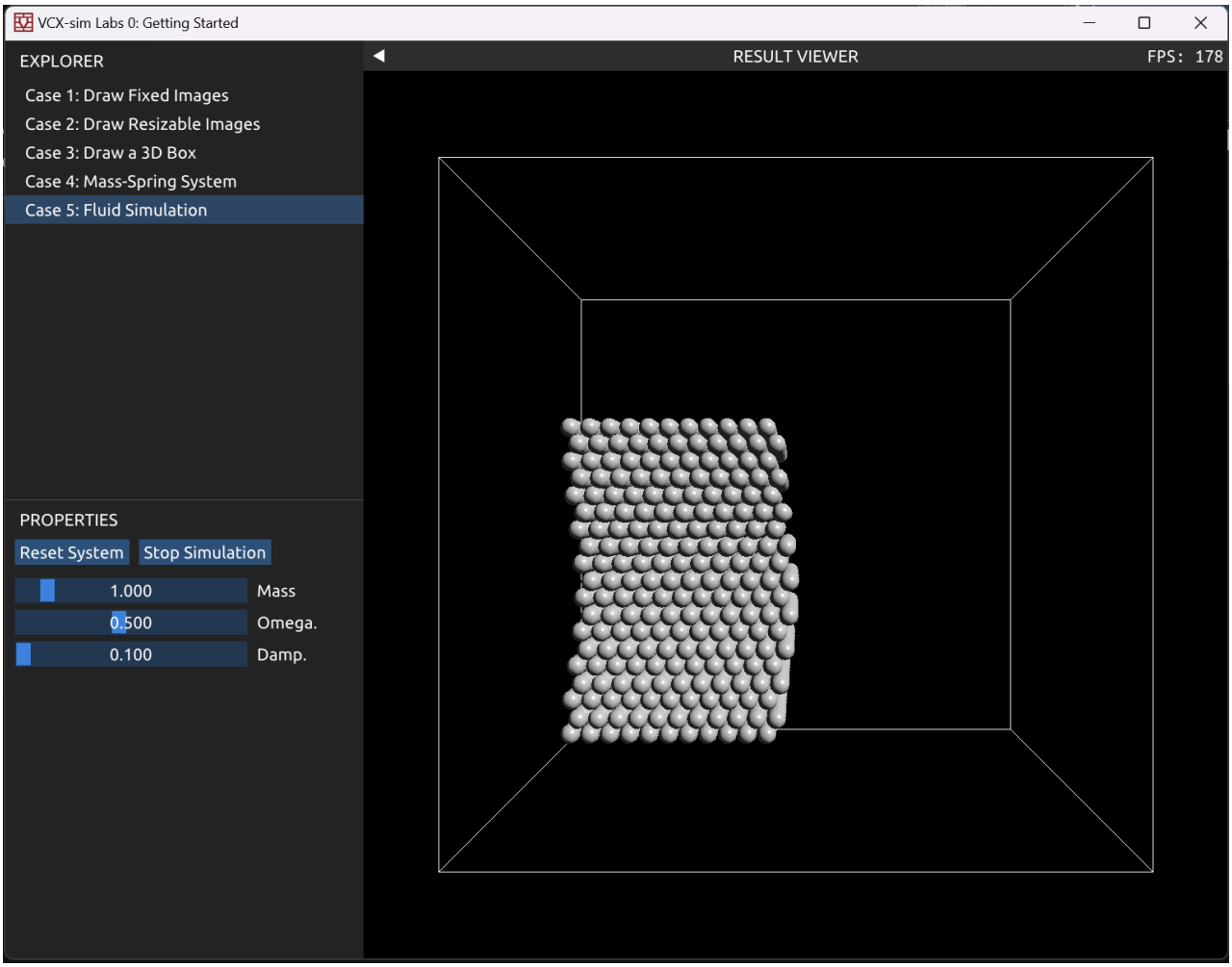
继续执行 `xmake run lab0` ，如果一切顺利，你会看到一个界面，通过界面可以切换显示方框中的五个 Case。


VCX-sim Labs 0: Getting Started
EXPLORER
Case 1: Draw Fixed Images
Case 2: Draw Resizable Images
Case 3: Draw a 3D Box
Case 4: Mass-Spring System
Case 5: Fluid Simulation
PROPERTIES
Zoom Tooltip
Small (320 x 320)
Size
White
Background
RESULT VIEWER
FPS: 32


VCX-sim Labs 0: Getting Started
EXPLORER
Case 1: Draw Fixed Images
Case 2: Draw Resizable Images
Case 3: Draw a 3D Box
Case 4: Mass-Spring System
Case 5: Fluid Simulation
PROPERTIES
Zoom Tooltip
RESULT VIEWER
FPS: 32


VCX-sim Labs 0: Getting Started
EXPLORER
Case 1: Draw Fixed Images
Case 2: Draw Resizable Images
Case 3: Draw a 3D Box
Case 4: Mass-Spring System
Case 5: Fluid Simulation
RESULT VIEWER
FPS: 60
PROPERTIES
Appearance
R:121
G:207
B:171
Box Color
1.000
x
2.000
y
3.000
z
0.000
pos_x
0.000
pos_y
0.000
pos_z


VCX-sim Labs 0: Getting Started
EXPLORER
Case 1: Draw Fixed Images
Case 2: Draw Resizable Images
Case 3: Draw a 3D Box
Case 4: Mass-Spring System
Case 5: Fluid Simulation
RESULT VIEWER
FPS: 32
PROPERTIES
Algorithm
Reset System
Start Simulation
1.000
Part. Mass
100.000
Spr. Stiff.
0.200
Spr. Damp.
0.300
Gravity
Appearance



Case 1 和 Case 2 展示了二维作图，以及图形学常见的画三角形。

Case 3 展示了如何渲染一个三维盒子，你可以基于此完成刚体的 Lab。同时 Case 3 还实现了简单的交互，除了使用鼠标左右键以及WASDEQ移动相机以及滚轮缩放，Case 3 还实现了按住 Alt 键用鼠标左键拖动物体。你可以基于此实现自己的交互。

Case 4 展示了隐式弹簧质点模型，你可以借鉴这个例子中的仿真流程。

Case 5 展示了流体的渲染，这个例子中流体粒子做简单的简谐运动。现有基础的流体场景搭建，你需要在lab2中更改 `Simulator` 进行对应算法的实践。

在编译过程中，你可能遇到一些问题。我们为常见的问题提供了 xmake FAQ（见后文）。如果它无法解决你的问题，欢迎联系课程助教。

4 - Prepare for IDEs

接下来，你可能会想要让你喜欢的 IDE 了解 xmake 项目，以提供智能提示，调试器集成等功能：

- Visual Studio: 执行 `xmake project -a x64 -k vsxmake ./build` 。 你会在 `vcx/build/vsxmake20xx` 目录下找到 .sln 解决方案文件。
- XCode: 执行 `xmake project -k xcode ./build` 。 你会在 `vcx/build/xcode` 目录下找到 XCode 项目文件。
- VS Code（图形化配置方案）：
  i. 首先执行 `xmake project -k compile_commands ./.vscode` 。
  ii. 安装 C/C++ 插件与 XMake 插件。
  iii. 选中顶部的 `View -> Command Palette...` ，输入 `XMake:` ，选择 `XMake: Update Intellisense` 。
  iv. 选中顶部的 `View -> Command Palette...` ，输入 `C/C++:` ，选择 `C/C++: Edit Configurations (UI)` 。

v. 选中 `C++ standard` 项，修改设置为 `C++20` 。

vi. 拉到最下方，点开 `Advanced Settings` ，在 `Compile Commands` 一栏输入 `${workspaceFolder}/.vscode/compile_commands.json`

vii. 返回 cpp 文件，现在 VS Code 应该已经能提供智能提示等功能了。

- VS Code（命令行配置方案）： 首先执行 `xmake project -k compile_commands ./.vscode` 。 然后你需要在 `vcx/.vscode` 目录下新建一个名为 `c_cpp_properties.json` 的文件，并写入：

```
{ "configurations": [ { "name": "Default", "compileCommands":
"${workspaceFolder}/.vscode/compile_commands.json" } ], "version": 4 }
```

这样 VS Code 就能找到刚才命令生成的 `compile_commands.json` 并用它来帮助理解 C++ 项目了。

xmake FAQ

- Q. 在 Windows 的 PowerShell 中输入 `xmake` 后显示「xmake : 无法将"xmake"项识别为 cmdlet、函数、脚本文件或可运行程序的名称」怎么办?
- A. 确保你已经通过 powershell 安装了 xmake；如果是通过下载 exe 安装包安装的 xmake，需要按照官网说明手动配置环境变量。
- Q. 在命令行中输入 `xmake` 后显示「note: xmake.lua not found, try generating it」怎么办?
- A. 确保命令行当前所在的文件夹是 `vcx-sim-master` ，如果不是，通过 cd 指令转到正确的路径。
- Q. 首次在命令行中输入 `xmake` 后需要安装一些包，此时报错下载失败并且能够看到「we can also download these packages manually」及「error: curl: (56) Recv failure: Connection was reset」字样怎么办?
- A. 这是因为访问 Github 时遇到了网络问题，有两种解决方案：
  - 打开本地代理，使用命令行设置好环境变量 `HTTPS_PROXY="127.0.0.1:<port>"` ，之后在命令行中运行xmake
    - 例如，当你使用wallesspku时，默认端口一般是7890，此时在命令行中运行：
    - MacOS/Linux

      ```
      HTTPS_PROXY="127.0.0.1:7890" xmake
      ```

    - Windows cmd

      ```
      set HTTPS_PROXY=127.0.0.1:7890
      xmake
      ```

    - Windows Powershell

      ```
      $env:HTTPS_PROXY = "127.0.0.1:7890"
      xmake
      ```

  - 可运行 `xmake g --pkg_searchdirs=<download-dir>` 并根据报错提示，手动下载软件包并重命名为指定名字
    - 一般情况，下载的软件包不用改名就可以识别到。一种方便的方式是将浏览器的默认下载路径加入搜索路径，例如如果在 Windows 上使用 Edge 浏览器，则默认下载路径为 `C:\Users\<username>\Downloads` ，设置

```
xmake g --pkg_searchdirs=C:\Users\<username>\Downloads
```

这样在 VSCode 自带终端等比较智能的终端中，按住 Ctrl+单击报错信息提示的下载url 跳转到浏览器下载，等一小会重新运行 xmake 即可。这种方法适用于使用浏览器插件作为代理或者不喜欢每次在命令行进行设置的同学。

- Q. 在命令行中输入 `xmake` 报错找不到编译器或「cannot get program for cxx」怎么办？？
- A. xmake 在各平台会默认使用该平台原生工具链，例如 Windows 上的 Visual Studio，MacOS 上的 XCode，Linux 上的 GCC，而对其他的工具链会报错找不到编译器。如果要使用 Msys2 提供的GCC编译器，在 Windows 上运行：

  ```
  xmake f -p mingw
  ```

  再执行编译步骤。如果要使用 Clang 编译器，运行

  ```
  xmake f --toolchain=clang
  ```

  再执行编译步骤。
- Q. 在 Windows 的 powershell 中输入 `xmake` ，提示找不到 Visual Studio，但是我明明安装了怎么办?
- A. 首先确保 Visual Studio 的安装路径不含中文，如果含中文需要重新安装到其他路径。其次打开 Visual Studio Installer，找到 `使用 C++ 的桌面开发` ，在可选项中查看 `对 v1xx 生成工具(最新)的 C++/CLI 支持` 一项是否勾选，如果未勾选，安装这一项并重新尝试编译。
- Q. 我的系统上安装了多个编译器，怎样指定使用哪一个编译器编译?
- A. xmake 对编译器提供了全局缓存。如果你新安装了一个编译器，需要使用 `xmake g -c` 清理全局缓存，再在项目目录运行 `xmake f -c` 重新探测编译器。对不同的编译器，有不同指定版本的方式。例如，指定使用 Visual Studio 2022：

  ```
  xmake f --vs=2022
  ```

  指定使用gcc11：

  ```
  xmake f --toolchain=gcc-11
  ```
- Q. 我使用 Mac OS 系统，安装时报错 `invalid Darwin version number: macos 12.3`
- A. 使用的 XCode 版本过低，将 XCode 更新到最新版本即可。

写在后面

到这里，你应该已经能够编译运行我们提供的 Lab 代码，并能够在舒适的开发环境中进行开发了。遇到问题的同学请尽快向助教提出你的疑问。

本次 lab0 中，大家只需要完成上述环境的配置即可。本文档的后半部分是一个简单导引，帮助大家理解示例代码，介绍其中需要模仿的关键模块；在后续的 lab 中，同学们可以搬运 lab0 的可视化、交互部分代码并进行自己的修改。大家可以提前阅读并熟悉我们的代码框架，或者在以后写 lab 的时候再进行参考。写过可视计算与交互概论课程大作业的同学可以跳过这部分内容。